# ApplyFlow — `spec.md` (V1)

## 1. Product Overview

### 1.1 Product Name

**ApplyFlow**

### 1.2 Product Type

A personal web application for **tracking internship/job application progress**, recruitment events, reminders, and “attention needed” cases.

### 1.3 Product Goal

ApplyFlow helps a user manage their internship/job application pipeline in one place. It is **not** a job board and **not** a recruiting platform. It does **not** submit applications on behalf of the user.

Its purpose is to help the user answer four questions clearly:

1. **Where have I applied?**
2. **What is the current state of each application?**
3. **What events happened or are scheduled next?**
4. **Which applications need my attention right now?**

### 1.4 Product Focus

ApplyFlow V1 focuses on three things:

1. **Application tracking**

2. company, role, status, source, notes, follow-up date

3. **Recruitment timeline tracking**

4. applied, HR call, OA, interview, follow-up, offer, rejection, notes

5. **Reminder / attention support**

6. no response after applying

7. no response after interview
8. overdue follow-up
9. upcoming events

### 1.5 Target User

Primary target user:

- students applying for internships
- new graduates applying for entry-level roles
- individuals who want a personal tracker for applications and recruitment progress

### 1.6 V1 Philosophy

V1 should be **small, useful, and finishable**. It should feel like a serious personal tool, not a toy CRUD app, but it must avoid turning into:

- a full ATS
- a CRM
- an email platform
- a team collaboration product

---

# 2. Product Scope

## 2.1 In Scope for V1

ApplyFlow V1 must support:

### A. Authentication

- register
- login
- get current user

### B. Application management

- create application
- list applications
- search applications
- filter applications by status
- view application details
- update application
- delete application

### C. Recruitment event tracking

- add events to an application
- view timeline of events for an application
- update event
- delete event

### D. Reminder and attention support

- set application follow-up date

- detect overdue follow-up
- detect no response after apply
- detect no response after interview
- detect upcoming scheduled events

### E. Dashboard

- total application count
- counts by current status
- upcoming events
- attention-needed applications

---

## 2.2 Out of Scope for V1

ApplyFlow V1 must **not** include:

- applying to jobs directly from the app
- recruiter/company-side functionality
- email inbox integration
- automatic parsing of email responses
- AI-generated CVs, cover letters, or interview answers
- file upload for CVs / documents
- team collaboration / shared applications
- kanban board / drag-and-drop workflow
- calendar month view
- push notifications / realtime notifications
- full contact management system
- user-configurable reminder thresholds
- browser extension or job scraping automation

---

# 3. Core Domain Concepts

ApplyFlow is built around **three main concepts**:

## 3.1 Application

An **Application** represents one internship/job opportunity for a specific company and role.

Examples:

- Frontend Intern at Company A
- Backend Intern at Company B

An application stores:

- core job/application info
- one current overall status

- optional follow-up reminder date
- a timeline of recruitment events

---

## 3.2 Recruitment Event

A **Recruitment Event** represents something that happened or is scheduled to happen in the recruitment process for a specific application.

Examples:

- applied
- HR call
- OA / assignment
- interview
- follow-up
- offer
- rejection
- note

Events are used to record:

- what happened
- when it happened
- what is scheduled next
- who contacted the user
- how the interview/call is conducted
- notes and logistics

---

## 3.3 Attention Flag

An **Attention Flag** is a system-generated signal that tells the user an application may need attention.

Examples:

- no response after applying for 14 days
- no response after interview for 7 days
- follow-up overdue
- interview in 2 days

Attention flags are not manually created by the user in V1.

---

# 4. Status Model

Each application has exactly one **current overall status**.

Allowed values for `currentStatus`:

- `saved`
- `applied`
- `in_process`
- `offer`
- `rejected`
- `withdrawn`

## 4.1 Meaning of Each Status

`saved`

The user saved the opportunity but has **not applied yet**.

`applied`

The user has applied, but there is **no meaningful recruitment progress yet**.

`in_process`

The application is actively moving through the recruitment process, such as:

- HR call
- OA
- interview
- post-interview waiting

`offer`

The user received an offer.

`rejected`

The company rejected the user.

`withdrawn`

The user decided to stop pursuing the application.

---

## 4.2 Important Rule About Status vs Events

`currentStatus` is an **application-level summary field**.
Detailed progress belongs to the **event timeline**.

Examples:

- “Technical interview scheduled on July 10” → event
- “Current state of this application is still active and in progress” → currentStatus

# 5. User Stories

## 5.1 Authentication

- As a user, I want to create an account so that my application data is stored privately.
- As a user, I want to log in and access my own tracker.
- As a user, I want only my own applications and events to be visible to me.

## 5.2 Application Tracking

- As a user, I want to create an application entry so I can track a company and role.
- As a user, I want to edit an application's current status so I know its overall state.
- As a user, I want to store notes and follow-up dates for each application.
- As a user, I want to delete applications I no longer need.

## 5.3 Recruitment Timeline

- As a user, I want to add events such as HR call, OA, interview, or follow-up so I can track the recruitment process.
- As a user, I want to store interview date, mode, location/link, and contact info.
- As a user, I want multiple events per application because a company may have multiple rounds.

## 5.4 Reminder / Attention

- As a user, I want the app to tell me when an application has been silent for too long after I applied.
- As a user, I want the app to tell me when I interviewed but still got no response after a while.
- As a user, I want to see which follow-ups are overdue.
- As a user, I want to see upcoming events in the next few days.

## 5.5 Dashboard / Search

- As a user, I want a dashboard that summarizes my application pipeline.
- As a user, I want to search by company or role.
- As a user, I want to filter by status to focus on a subset of applications.

---

# 6. Functional Requirements

# 6.1 Authentication

## 6.1.1 Register

The system must allow a user to register with:

- `displayName`

- `email`
- `password`

### 6.1.2 Login

The system must allow a user to log in with:

- `email`
- `password`

### 6.1.3 Current User

The system must allow the frontend to fetch the currently authenticated user.

### 6.1.4 Security Rules

The system must:

- hash passwords before storing them
- never return password hashes in API responses
- protect private routes with JWT authentication
- ensure users can only access their own data

---

## 6.2 Application Management

The system must allow a user to create an application with at least:

- company
- role
- currentStatus

Optional fields:

- jdUrl
- source
- notes
- followUpAt

The system must allow a user to:

- list all their applications
- search by company or role
- filter by currentStatus
- sort applications
- view application details
- update application fields
- delete application

The system must ensure:

- each application belongs to exactly one user
- users can only read/update/delete their own applications

---

# 6.3 Recruitment Event Management

The system must allow a user to add recruitment events to an application.

Supported event types in V1:

- `applied`
- `hr_call`
- `oa`
- `interview`
- `follow_up`
- `offer`
- `rejected`
- `note`

The system must allow a user to:

- create an event for an application
- list events for an application
- update an event
- delete an event

The system must ensure:

- an event belongs to exactly one application
- an event belongs to the same user as the application

---

# 6.4 Reminder / Attention Support

The system must generate attention-related information based on application and event data.

There are **two categories** of attention logic in V1:

## A. Silence Flags

Flags about waiting too long for company response:

- `NO_RESPONSE_AFTER_APPLY`
- `NO_RESPONSE_AFTER_INTERVIEW`

### B. Reminder / Schedule Flags

Flags about dates the user should care about:

- `FOLLOW_UP_OVERDUE`
- `UPCOMING_EVENT`

Detailed rules are defined in Section 13.

---

## 6.5 Dashboard

The dashboard must show:

### A. Summary section

- total applications
- count by currentStatus

### B. Upcoming events section

- events scheduled within the next 3 days

### C. Attention-needed section

Applications flagged for:

- no response after apply
- no response after interview
- overdue follow-up

---

## 6.6 Search / Filter / Sort

The application list must support:

- text search by company or role
- filter by currentStatus
- sort by created time or updated time

Pagination is optional in V1 but recommended if easy to implement.

---

# 7. Event Model

## 7.1 Supported Event Types

ApplyFlow V1 supports these event types:

- `applied`
- `hr_call`
- `oa`
- `interview`
- `follow_up`
- `offer`
- `rejected`
- `note`

---

## 7.2 Common Event Fields

Every event should support the following common fields:

- `_id`
- `applicationId`
- `userId`
- `type`
- `title`
- `occurredAt` (optional)
- `scheduledAt` (optional)
- `note` (optional)
- `createdAt`
- `updatedAt`

Optional communication fields:

- `contactName`
- `contactPhone`
- `contactEmail`

Optional logistics fields:

- `mode` — one of:
- `online`
- `offline`
- `phone`
- `location`
- `meetingLink`

---

## 7.3 Event Type Guidelines

`applied`

Represents the actual application submission.

Typical fields:

- `occurredAt`
- optional note

---

`hr_call`

Represents an HR call or screening contact.

Typical fields:

- `scheduledAt` or `occurredAt`
- `mode` (often `phone` or `online`)
- contact info
- note

---

`oa`

Represents an online assessment or assignment.

Typical fields:

- `scheduledAt`
- note
- optional meeting link / platform note

---

`interview`

Represents an interview round.

Typical fields:

- `scheduledAt`
- `occurredAt`
- `mode`
- `location` or `meetingLink`
- contact info
- note

For V1, interview round labels such as “Round 1”, “Technical”, “Final” can live in:

- `title`
- or `note`

A dedicated `round` field is intentionally deferred.

---

### `follow_up`

Represents an actual follow-up action taken by the user, such as:

- sending a follow-up email
- messaging HR
- asking for update after interview

Typical fields:

- `occurredAt`
- contact info
- note

---

### `offer`

Represents receiving an offer.

Typical fields:

- `occurredAt`
- note

---

### `rejected`

Represents receiving a rejection.

Typical fields:

- `occurredAt`
- note

---

### `note`

Represents a free-form timeline note.

Typical fields:

- optional `occurredAt`

- note

---

# 8. Data Model

ApplyFlow V1 should use at least these collections:

- `users`
- `applications`
- `application_events`

---

# 9. Collection Schemas

## 9.1 `users` Collection

Suggested document shape:

```json id="9vf8e2" { "_id": "ObjectId", "displayName": "string", "email": "string", "passwordHash": "string", "createdAt": "Date", "updatedAt": "Date" }
```

```
### Rules
- `email` must be unique
- `passwordHash` must never be exposed to the client

---

## 9.2 `applications` Collection

Suggested document shape:

```json id="n1p4kt"
{
  "_id": "ObjectId",
  "userId": "ObjectId",

  "company": "string",
  "role": "string",
  "jdUrl": "string | null",
  "source": "string | null",
  "notes": "string | null",

  "currentStatus": "saved | applied | in_process | offer | rejected |
withdrawn",

  "followUpAt": "Date | null",
```
```

```
  "createdAt": "Date",
  "updatedAt": "Date"
}
```

### Required fields

- `userId`
- `company`
- `role`
- `currentStatus`
- `createdAt`
- `updatedAt`

### Optional fields

- `jdUrl`
- `source`
- `notes`
- `followUpAt`

---

## 9.3 `application_events` Collection

Suggested document shape:

```json id="s8ydz7" { "_id": "ObjectId", "applicationId": "ObjectId", "userId": "ObjectId",

"type": "applied | hr_call | oa | interview | follow_up | offer | rejected | note", "title": "string",

"occurredAt": "Date | null", "scheduledAt": "Date | null",

"mode": "online | offline | phone | null", "location": "string | null", "meetingLink": "string | null",

"contactName": "string | null", "contactPhone": "string | null", "contactEmail": "string | null",

"note": "string | null",

"createdAt": "Date", "updatedAt": "Date" }
```

```
### Required fields
- `applicationId`
- `userId`
- `type`
- `title`
- `createdAt`
- `updatedAt`

### Optional fields
```

```
- all time fields
- all contact fields
- all logistics fields
- note

---

# 10. Validation Rules

# 10.1 User Validation

## Register
Required:

- `displayName`: non-empty string
- `email`: valid email format
- `password`: minimum 8 characters recommended

## Login
Required:

- `email`
- `password`

---

# 10.2 Application Validation

## Create application
Required:

- `company`: non-empty string
- `role`: non-empty string
- `currentStatus`: one of:
  - `saved`
  - `applied`
  - `in_process`
  - `offer`
  - `rejected`
  - `withdrawn`

Optional:

- `jdUrl`: valid URL if provided
- `source`: string
- `notes`: string
- `followUpAt`: valid date if provided

## Update application
All fields optional, but if provided:
```

```
- `currentStatus` must be valid
- `followUpAt` must be a valid date or explicit null

---

# 10.3 Event Validation

## Create event
Required:

- `type`: valid event type
- `title`: non-empty string

Optional:

- `occurredAt`: valid date
- `scheduledAt`: valid date
- `mode`: `online | offline | phone`
- `location`: string
- `meetingLink`: string
- `contactName`: string
- `contactPhone`: string
- `contactEmail`: valid email if present
- `note`: string

## Update event
All fields optional, but if provided they must be valid.

---

# 11. Business Rules

## 11.1 Ownership
Every application belongs to exactly one user.
Every event belongs to exactly one user and one application.

All application/event queries must be scoped by authenticated `userId`.

---

## 11.2 Event Ownership Consistency
When creating an event for an application:

- the application must exist
- the application must belong to the authenticated user
- the event must store the same `userId` as the application owner

---

## 11.3 Current Status Is User-Controlled in V1
In V1, `currentStatus` is primarily controlled by the user rather than fully
```

```
auto-derived from events.

The UI may encourage status updates such as:

- after adding an `interview` event, the user may set `currentStatus =
in_process`
- after adding an `offer` event, the user may set `currentStatus = offer`
- after adding a `rejected` event, the user may set `currentStatus =
rejected`

But the backend does **not** need to auto-sync status from event types in V1.

---

## 11.4 Follow-Up Behavior
`followUpAt` and `follow_up` event are **different concepts**.

### `followUpAt`
An application-level reminder date representing:

- when the user plans to follow up
- or when the user wants to revisit the application

### `follow_up` event
A timeline record representing an actual follow-up action that already
happened.

These two must not be merged.

---

## 11.5 Deleting an Application
When an application is deleted:

- all events belonging to that application must also be deleted

V1 can use hard delete for simplicity.

---

## 11.6 Timeline Ordering
Application detail timeline should be displayed in chronological order.

Suggested ordering priority:
1. `occurredAt` if present
2. else `scheduledAt` if present
3. else `createdAt`

Frontend may choose ascending or descending presentation, but the rule must
be consistent.
```

```
---

# 12. Dashboard Requirements

The dashboard must show three main sections.

## 12.1 Status Summary
Display:

- total applications
- counts by:
  - saved
  - applied
  - in_process
  - offer
  - rejected
  - withdrawn

---

## 12.2 Upcoming Events
Display events scheduled within the next **3 days**.

Each item should show at least:

- event type
- title
- company
- role
- scheduled date/time

---

## 12.3 Attention Needed
Display applications with active attention flags:

- `NO_RESPONSE_AFTER_APPLY`
- `NO_RESPONSE_AFTER_INTERVIEW`
- `FOLLOW_UP_OVERDUE`

Each item should show:

- company
- role
- flag type or human-friendly message
- relevant date / why it is flagged

---

# 13. Attention Flag Logic
```

Attention flags are **computed**, not manually created by the user.

Each flag object returned to the frontend should include at least:

- `flagType`
- `applicationId`
- `company`
- `role`
- `message`
- `referenceDate`

---

## 13.1 Attention Flag Categories

ApplyFlow V1 uses **two categories** of attention logic.

### A. Silence Flags
These represent waiting too long for company response:

- `NO_RESPONSE_AFTER_APPLY`
- `NO_RESPONSE_AFTER_INTERVIEW`

### B. Reminder / Schedule Flags
These represent dates the user should care about:

- `FOLLOW_UP_OVERDUE`
- `UPCOMING_EVENT`

---

## 13.2 Eligibility Rule for Silence Flags

**Silence flags must only be evaluated for applications whose `currentStatus` is one of:**

- `applied`
- `in_process`

Applications with status:

- `saved`
- `offer`
- `rejected`
- `withdrawn`

must **not** receive:

- `NO_RESPONSE_AFTER_APPLY`
- `NO_RESPONSE_AFTER_INTERVIEW`

```
### Why
If an application is already:

- withdrawn,
- rejected,
- or otherwise no longer actively awaiting response,

showing “no response after apply/interview” would be misleading and bad UX.

---

## 13.3 Flag: `NO_RESPONSE_AFTER_APPLY`

### Purpose
Detect applications where the user applied but there has been no recorded
progress for too long.

### Conditions
Flag an application if **all** of the following are true:

1. `currentStatus` is either:

   - `applied`
   - `in_process`

2. the application has at least one `applied` event

3. take the **most recent** `applied` event

4. after that most recent `applied` event, there is **no later event** of
type:

   - `hr_call`
   - `oa`
   - `interview`
   - `offer`
   - `rejected`
   - `follow_up`

5. at least **14 days** have passed since that `applied` event’s effective
date

### Effective date for calculation
Use:
1. `occurredAt` if present
2. else `scheduledAt` if present
3. else do not generate this flag for that event because the reference date
is unreliable

### Example message
- “Applied 16 days ago but no response has been recorded yet.”
```

```
---

## 13.4 Flag: `NO_RESPONSE_AFTER_INTERVIEW`

### Purpose
Detect applications where the user interviewed but has not recorded any later
response or progress for too long.

### Conditions
Flag an application if **all** of the following are true:

1. `currentStatus` is either:

   - `applied`
   - `in_process`

2. the application has at least one `interview` event

3. take the **most recent** `interview` event

4. after that most recent interview event, there is **no later event** of
type:

   - `offer`
   - `rejected`
   - `follow_up`
   - `interview`

5. at least **7 days** have passed since that interview event’s effective
date

### Effective date for calculation
Use:
1. `occurredAt` if present
2. else `scheduledAt` if present
3. else do not generate this flag for that event

### Example message
- “Interview completed 8 days ago but no response has been recorded yet.”

---

## 13.5 Flag: `FOLLOW_UP_OVERDUE`

### Purpose
Detect applications whose follow-up date has passed.

### Conditions
Flag an application if:
1. `followUpAt` exists
```

```
2. `followUpAt` is earlier than the current date/time
3. `currentStatus` is **not** one of:

   - `rejected`
   - `withdrawn`

### Notes
- This flag can still apply to:
  - `saved`
  - `applied`
  - `in_process`
  - optionally `offer` if the product later decides offer follow-up matters
- For V1, a simple and safe rule is:
  - allow for `saved`, `applied`, `in_process`
  - do not show for `rejected` or `withdrawn`

### Recommended V1 implementation
Show `FOLLOW_UP_OVERDUE` only when `currentStatus` is one of:

- `saved`
- `applied`
- `in_process`

This keeps the UX simple and avoids awkward follow-up reminders for closed
cases.

### Example message
- “Follow-up date has passed.”

---

## 13.6 Flag / Section: `UPCOMING_EVENT`

### Purpose
Surface scheduled events happening soon.

### Conditions
An event is considered upcoming if:
1. `scheduledAt` exists
2. `scheduledAt` is within the next **3 days**
3. the parent application is not in a terminal/closed state where the event
is no longer relevant

### Recommended V1 rule
Only include upcoming events if application `currentStatus` is one of:

- `saved`
- `applied`
- `in_process`

Do not include if:
```

```
- `rejected`
- `withdrawn`

Handling of `offer` can be deferred, but for V1 it is simpler to exclude it
from upcoming events unless there is a strong product reason otherwise.

### UX note
`UPCOMING_EVENT` may be returned in the dashboard's "Upcoming Events" section
instead of mixing it into the same list as silence flags.

---

# 14. API Requirements

All API paths below assume a base prefix such as:

`/api/v1`

---

# 14.1 Auth APIs

## `POST /auth/register`
Create a new user account.

### Request body
```json id="fxm2w1"
{
  "displayName": "dazoriii",
  "email": "user@example.com",
  "password": "12345678"
}
```

### Response

Return:

- success message
- created user info without password hash

Auto-login after register is optional for V1. Simpler option:

- register only creates account
- user logs in separately

---

## `POST /auth/login`

Authenticate a user.
```

### Request body

```json id="h1rq8n" { "email": "user@example.com", "password": "12345678" }

```
### Response example
```json id="o5jv3c"
{
  "message": "Login successful",
  "accessToken": "jwt-token",
  "user": {
    "_id": "user-id",
    "displayName": "dazoriii",
    "email": "user@example.com"
  }
}
```

---

`GET /auth/me`

Return current authenticated user.

Requires:

- valid JWT

---

# 14.2 Application APIs

`POST /applications`

Create a new application.

### Request body example

```json id="k2m8sa" { "company": "OpenAI", "role": "Frontend Intern", "jdUrl": "https://example.com/job", "source": "LinkedIn", "notes": "Looks interesting", "currentStatus": "saved", "followUpAt": "2026-07-15T00:00:00.000Z" }

```
### Behavior
- application belongs to authenticated user
- server sets `createdAt` and `updatedAt`

---

## `GET /applications`
Get the authenticated user’s applications.
```
```

```
### Supported query params
- `search`
- `status`
- `sortBy`
- `sortOrder`

### Example
`GET /applications?
search=frontend&status=in_process&sortBy=updatedAt&sortOrder=desc`

### Response
Should return a list of applications.
Pagination metadata is optional in V1.

---

## `GET /applications/:applicationId`
Get one application detail.

### Response
At minimum, return the application document.

Recommended V1 design:

- `GET /applications/:applicationId` → application detail only
- `GET /applications/:applicationId/events` → timeline events separately

This keeps backend responsibilities clearer.

---

## `PATCH /applications/:applicationId`
Update an application.

Allowed fields:

- company
- role
- jdUrl
- source
- notes
- currentStatus
- followUpAt

---

## `DELETE /applications/:applicationId`
Delete an application and its events.

---
```

```
# 14.3 Event APIs

## `POST /applications/:applicationId/events`
Create a new event for an application.

### Request body example
```json id="g7zt4p"
{
  "type": "interview",
  "title": "Technical Interview",
  "scheduledAt": "2026-07-10T14:00:00.000Z",
  "mode": "online",
  "meetingLink": "https://meet.google.com/...",
  "contactName": "HR Nguyen",
  "contactEmail": "hr@example.com",
  "note": "Prepare React and JavaScript questions"
}
```

---

### `GET /applications/:applicationId/events`

Get timeline events for an application.

#### Response

Return events sorted chronologically according to the timeline ordering rule.

---

### `PATCH /applications/:applicationId/events/:eventId`

Update an event.

---

### `DELETE /applications/:applicationId/events/:eventId`

Delete an event.

---

# 14.4 Dashboard API

### `GET /dashboard/summary`

Return dashboard information for the authenticated user.
```

### Suggested response shape

```json id="c9u4mr" { "totalApplications": 11, "statusCounts": { "saved": 3, "applied": 5, "in_process": 2, "offer": 0, "rejected": 1, "withdrawn": 0 }, "upcomingEvents": [ { "eventId": "evt_1", "applicationId": "app_1", "company": "OpenAI", "role": "Frontend Intern", "type": "interview", "title": "Technical Interview", "scheduledAt": "2026-07-10T14:00:00.000Z" } ], "attentionFlags": [ { "flagType": "NO_RESPONSE_AFTER_APPLY", "applicationId": "app_2", "company": "Company A", "role": "Backend Intern", "message": "Applied 15 days ago but no response has been recorded yet.", "referenceDate": "2026-06-17T00:00:00.000Z" } ] }
```

```
---

# 15. Frontend Page Requirements

ApplyFlow V1 should include at least these pages.

## 15.1 Login Page
Must include:

- email field
- password field
- login action
- link to register page

---

## 15.2 Register Page
Must include:

- display name
- email
- password
- register action

---

## 15.3 Dashboard Page
Must show:

- status summary cards
- upcoming events
- attention-needed applications

---

## 15.4 Applications Page
Must show:

- application list
- search input
```

```
- status filter
- button to create application
- edit/delete actions

The list can be table-based or card-based.
V1 does not need a kanban board.

---

## 15.5 Application Detail Page
Must show:

### A. Application Overview
- company
- role
- current status
- source
- jdUrl
- followUpAt
- notes

### B. Recruitment Timeline
- list of events in chronological order
- add event action
- edit event action
- delete event action

### C. Optional Attention Display
If the application currently has relevant flags, they may be shown on the
detail page too.

---

# 16. Error Handling Requirements

The backend should return consistent JSON error responses.

Recommended response shape:

```json id="t7r3ma"
{
  "message": "Validation failed",
  "errors": {
    "company": "Company is required"
  }
}
```

Or for general errors:
```

```
json id="m6s9kd"
{
  "message": "Application not found"
}
```

Minimum error cases to handle:

- invalid input
- unauthorized request
- forbidden access to another user's data
- resource not found
- invalid ObjectId / malformed id
- internal server error

---

# 17. Suggested Default Thresholds

These values are fixed for V1:

- **No response after apply** → 14 days
- **No response after interview** → 7 days
- **Upcoming event window** → next 3 days

These should be defined in a central constants/config layer so they can be changed later without rewriting business logic.

---

# 18. Suggested Milestones

## Milestone 1 — Foundation

- project setup
- environment config
- MongoDB connection
- auth routes and middleware
- frontend routing and auth flow

## Milestone 2 — Applications CRUD

- create/list/detail/update/delete applications
- search/filter/sort

## Milestone 3 — Event Timeline

- create/list/update/delete events
- application detail timeline UI

### Milestone 4 — Dashboard & Attention Logic

- status summary
- upcoming events
- no-response flags
- overdue follow-up flags

### Milestone 5 — Cleanup

- validation
- error handling
- loading states
- README
- sample seed data if needed

---

# 19. Open Decisions Intentionally Deferred

These decisions are intentionally **not finalized** in V1:

- whether `currentStatus` should auto-sync from event types
- whether a `follow_up` event should automatically clear an overdue follow-up reminder
- whether interviews need a dedicated `round` field
- whether applications should have a primary contact field in addition to event-level contacts
- whether reminder thresholds should become user-configurable
- whether offer-specific follow-up logic should exist
- whether email integration should be added later

These should not block V1 implementation.

---

# 20. Final Summary

ApplyFlow V1 is a **personal internship recruitment tracker** with three core capabilities:

### 1. Application tracking

Store:

- company
- role
- status
- source
- notes
- follow-up date

## 2. Recruitment timeline tracking

Track:

- applied
- HR call
- OA
- interview
- follow-up
- offer
- rejection
- note

## 3. Reminder / attention support

Surface:

- applications silent too long after apply
- applications silent too long after interview
- overdue follow-up
- upcoming scheduled events

The product is successful if the user can open it and quickly understand:

- which companies they applied to
- what stage each application is in
- what interviews or follow-ups are coming up
- which applications need attention right now